

CS 498 Final Report

Nathan Ma

May 12, 2026

Introduction

As hardware improves and computational power overall increases, machine learning models are growing increasingly more complex as well. One way this growth is reflected is in the sheer number of parameters that recent models have compared to their older counterparts. For example, the GPT-3 model is estimated to have around 175 billion parameters, the GPT-4 model is estimated to have 10x the amount at 1.7 trillion parameters, and the GPT-5 model likely has an even higher amount.

However, this roughly exponential growth of parameters cannot be discussed without mentioning the increasing complexity of model architecture as well. Sticking with the example of the GPT models, the GPT-2 model consists of a standard feed forward neural network with transformer blocks whereas the recent GPT-5 model is widely believed to employ a mixture of experts approach, which essentially means that (and this is a massive oversimplification) GPT-5 employs several GPT-2 like architectures all within the same model.

Consequently, the already relevant challenge of efficient model training has become all the more important. An old saying referring to boxing goes: "the bigger they are, the harder can fall", yet this idea can just as easily be applied to our model training problem. With so many parameters, how do we ensure our model avoids overfitting and actually learns meaningful features about the data?

Similar to advancements in architecture for language models such as GPT, image recognition models have also seen much growth in recent years. Most notably, vision transformer (ViT) models have overtaken convolutional neural networks (CNN) as the state of art image classification method. As the name suggests, CNNs use layers of convolutions to learn features from a more granular level towards a more broad understanding. On the other hand, ViT models use a lot of the ideas from language transformer models in an image setting: images are broken into patches, those patches are tokenized, and position is incorporated through a learned vector.

Thus, in this report, we investigate a promising approach to this question of how to efficiently and effectively train a vision transformer model.

Background

Masked token prediction trains models to predict missing words in sentences. This motivates models to understand contextual relationships between words, and, as a result, masked token prediction is often used as a pretraining task to achieve a baseline language ability before more specialized fine-tuning [2].

In the paper Masked Autoencoders are Scalable Vision Learners, the authors propose an analogous approach towards pretraining vision transformer models [1]. Instead of masking words in a sentence, we now mask patches in an image. Through their experiments, the authors found that masking a relatively high percentage (75%) of the input image provided a nontrivial and meaningful self-supervised learning task, allowing the vision transformer model to learn useful features towards the objective of image reconstruction.

Here is an overview of pretraining architecture suggested by the authors:

1. **Masking:** An image is divided into regular non-overlapping patches. For example, images from the ImageNet-1K dataset have a resolution of 224x224. We use a patch size of 16x16, which creates a total of 196 patches. Then, a subset of the patches is randomly sampled and the rest are masked (removed). We choose a high masking ratio to eliminate redundancy and make it so that the image reconstruction task is not solved by simply extrapolating from nearby patches.
2. **Encoder:** The subset of patches is then passed into an encoder, which follows a standard vision transformer architecture. Patches are embedded through linear projection with added positional embeddings, and the result is fed through a series of transformer blocks.
3. **Decoder:** The decoder takes the output from the encoder alongside the mask tokens. This combined input is fed through a similar but shallower vision transformer architecture. The final output from the decoder can be reshaped to form the full predicted image, which can be compared with the original image to complete the training loop.

The main power in this approach comes from the decision to mask input patches at a high rate. Doing so forces the model to learn meaningful cues for image reconstruction, and also greatly reduces the computation within the encoder. As a result, pretraining will take less time since we don't need to consider the full set of patches, and finetuning will take less time because the encoder has already done a majority of the learning. Thus, by employing this masking strategy in pretraining, we effectively get the best of both worlds.

Experiments

We first replicated the masked autoencoder on the smaller CIFAR-10 dataset to get a rough feel for potential performance on the image reconstruction task.

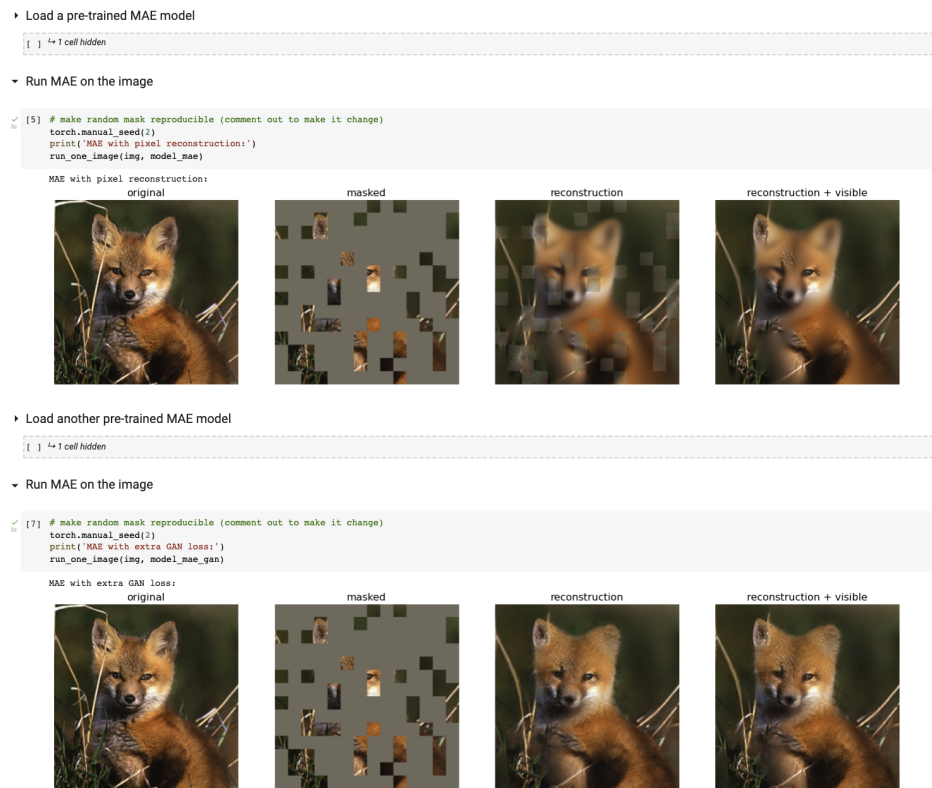


Figure 1: Sample result from original paper repository

The results from the paper looked pretty impressive, so we wanted to see if something similar could be achieved on a smaller scale. In our approach, we used smaller dimensions for our encoder and decoder, and we also used fewer transformer blocks. We trained for 80 epochs, and each epoch took approximately 30 seconds to complete. All in all, it was a very quick pretraining process.

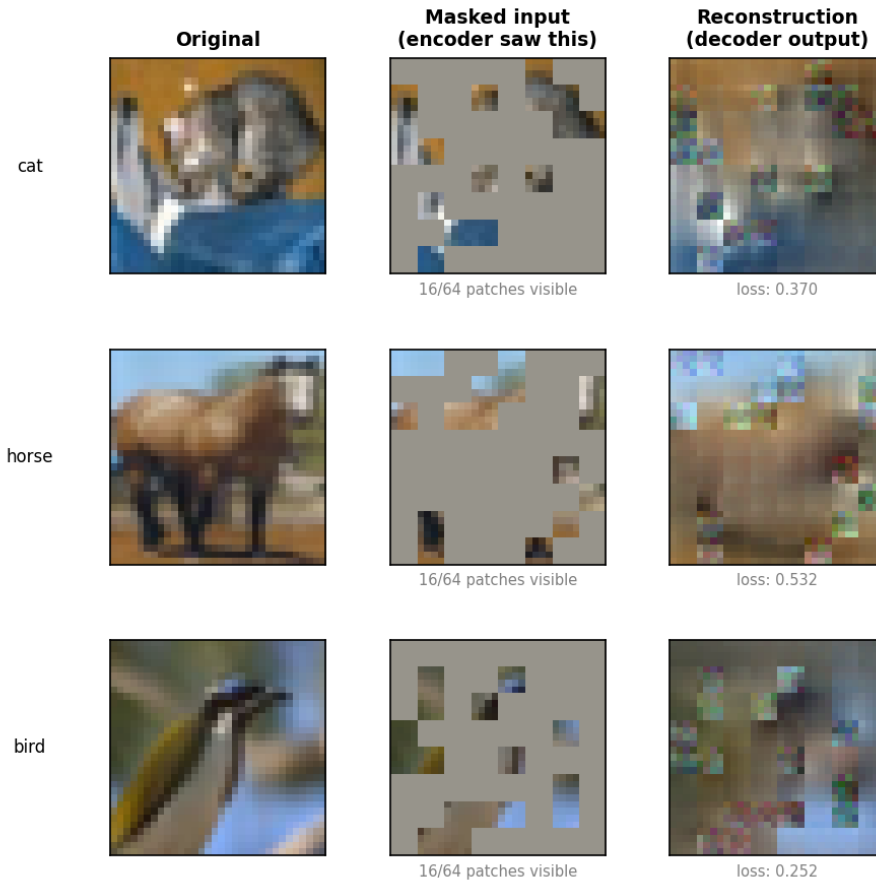


Figure 2: Image reconstruction from CIFAR-10 dataset

Unfortunately, the results were not too promising. As we can see, the reconstructed images are mostly incoherent and bear little resemblance to the images they were originally derived from. Later in the discussion section, we will talk about some potential factors contributing to this result, but, at the very least, we now have a (rather poor) baseline on which we can compare when we pretrain our masked autoencoder on images from the ImageNet-1K dataset.

We trained our full masked autoencoder class with the following set of hyperparameters:

Optimizer	AdamW
Base learning rate	1.5e-4
Weight decay	0.05
Optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.95$
Batch size	4096
Learning rate schedule	Cosine decay
Epochs	80
Augmentation	RandomResizedCrop

For the vision transformer, we used the following architecture:

Encoder dimension	768
Encoder heads	12
Encoder blocks	12
Transformer MLP ratio	4

For the most part, we stayed true to the paper, but we had to drastically cut the number of epochs from 1600 to 80. I found that a training a single epoch varied from 10 to 20 minutes. Training 1600 epochs would take over 200 hours worst case, which would be well above the 48 hour limit on job runs that the NCSA Delta system enforces. Therefore, I decided on the number of 80 hours somewhat arbitrarily, perhaps consistent with the number of epochs when I trained on CIFAR-10 data. Additionally, our vision transformer architecture follows the ViT-Base architecture. Although the paper also implemented the ViT-Large and ViT-Huge architectures, we chose to just stick with the ViT-Base architecture because the difference in performance was not super significant and doing so would also avoid exceedingly high job times. Finally, for similar reasons, we also trained our models on a random 100 class subset of the ImageNet-1K data.

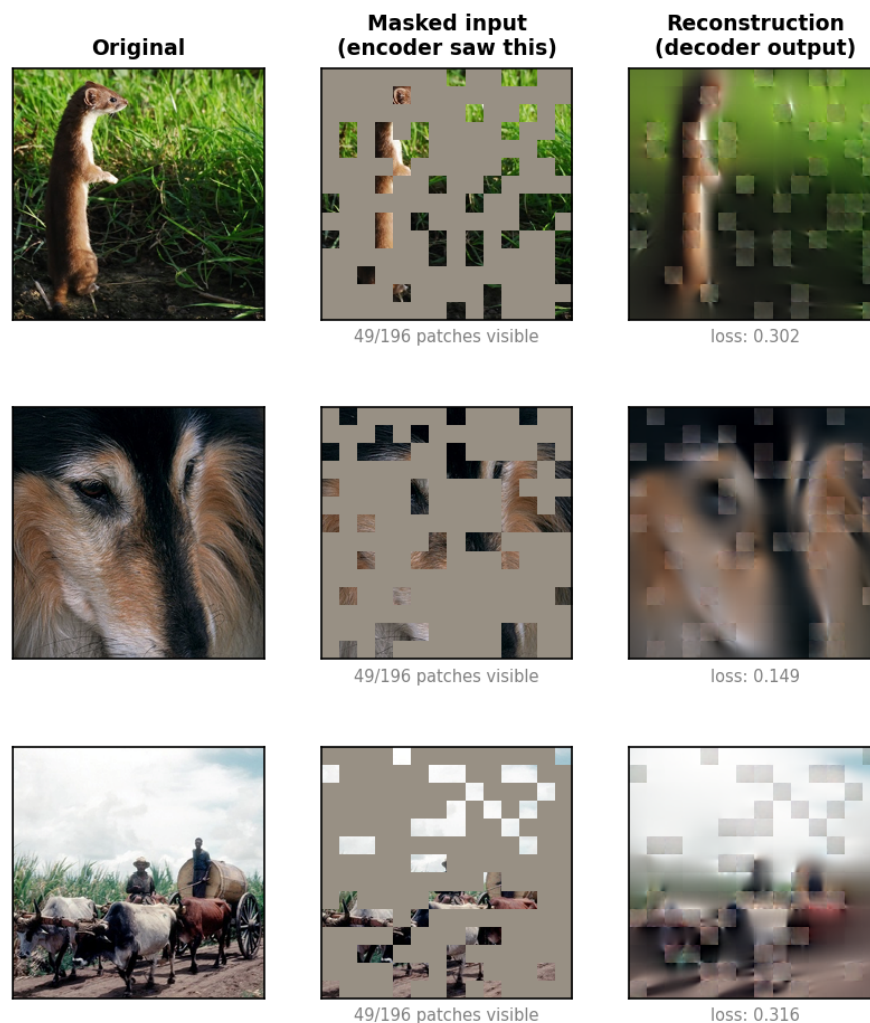


Figure 3: Image reconstruction from Imagenet-1K dataset

As we can see the resultant output is not perfect, but it at least looks somewhat recognizable. This is a big step up from the jumbled mess that was our previous attempt at reconstruction!

Putting everything together, we transferred the learned encoder weights from the MAE class to our vision transformer. We evaluated the effectiveness of the MAE in two ways.

1. **Training Speedup:** Theoretically, the MAE should have learned meaningful image reconstruction features. As a result, if we directly transfer the weights, this should give the new model a "head start" and we should see faster convergence to a similar training accuracy.
2. **Linear Probing:** If we freeze the weights in the transformer blocks and only do gradient descent on the classification head, how good can the resulting accuracy be? In theory, somewhat high because, again, we expect the MAE to have learned meaningful features.

The final results are shown in the table below:

Baseline ViT	ViT w/ MAE weights	Linear Probing
76.3%	73.9%	39.7%

For reference, we trained the baseline ViT for 800 epochs and two MAE-assisted models for only 200 epochs. The results of linear probing are strong and comparable with that of the paper. Furthermore, in the case of the model initialized with weights from the pretrained MAE, we see comparable accuracy with only a fourth of the epochs.

Discussion

First, we will address why we observed such a disappointing output when pretraining on the CIFAR-10 dataset. For context, images in the CIFAR-10 dataset have a resolution of 32x32 compared to the 224x224 resolution of ImageNet-1K images. To adjust, we used 4x4 patches, which resulted in 64 total patches. As a result, our encoder is receiving less patches and less pixels per patch. Perhaps there is just not enough input and the complexity of our model architecture is overkill.

Another surprising result was how the MAE pretraining time on the ImageNet-1K dataset was roughly 30x as long as the CIFAR-10 dataset. Investigating further, the ImageNet-1K dataset is around 8x as large as CIFAR-10 (I was only able to load 440K images before running into an error), and there are roughly 3x as many patches (196 vs 64). Already, we would expect a roughly 72x (attention scales quadratically with sequence length) slowdown. Not to mention, the ViT-Base architecture also has a higher embedding dimension and more transformer blocks. So it's actually surprising that pretraining the MAE on the ImageNet-1K dataset turned out relatively quickly compared to the CIFAR-10 dataset.

Finally, if you look at the training log for the ViT pretrained with MAE weights, you'll see that it starts at a roughly similar accuracy to the baseline model. This was also a bit surprising because one would expect the pretrained model to start with higher accuracy. Doing some research online, the MAE constructs its latent space such that different classes have different embeddings [4]. As a result, fine tuning starts at a similar level to training from scratch because distinctions in the representations aren't yet learned. However, since those distinctions do exist, fine tuning is accelerated, which is what we see here.

Overall, I'm pretty happy with how this final project turned out. I got some good experience building the training pipelines, and it was very cool seeing the image reconstruction output. All code can be found in the linked Github repository [3]. Thanks for reading!

References

- [1] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15979–15988, 2022.
- [2] Eda Kavlakoglu Jacob Murel. What are masked language models? <https://www.ibm.com/think/topics/masked-language-model>.
- [3] Nathan Ma. Reference code. <https://github.com/amingma/mae-small>.
- [4] Anika Shrivastava, Renu Rameshan, and Samar Agnihotri. Robust representation learning in masked autoencoders, 2026.